1. Declaração do Documento e Metadados (<head>)

```HTML
<!DOCTYPE html>
<html lang="pt-BR">
```

- **<!DOCTYPE html>**: Informa ao navegador que o documento deve ser interpretado conforme o padrão moderno **HTML5**.
- **<html lang="pt-BR">**: Tag raiz da página. O atributo lang="pt-BR" declara que o conteúdo está em português do Brasil, auxiliando mecanismos de busca (SEO) e leitores de tela para acessibilidade.

```HTML
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>BlogPy - Projeto Didático de Programação Web</title>
```

- **<meta charset="utf-8">**: Define a codificação universal de caracteres (UTF-8), garantindo que acentuação, cedilhas e caracteres especiais não fiquem corrompidos.
- **<meta name="viewport" content="width=device-width, initial-scale=1.0">**: Tag essencial para **design responsivo**. Ela instrui o navegador do celular ou tablet a ajustar a largura da página à largura física do dispositivo, com escala inicial de 100%.
- **<title>**: Define o título exibido na aba do navegador e nos favoritos.

```HTML
  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@600;7
00&family=Roboto:wght@400;500&display=swap" rel="stylesheet">
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/boots
trap.min.css" rel="stylesheet">
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bo
otstrap-icons.min.css" rel="stylesheet">
```

- **Google Fonts**: Importa remotamente as famílias de fontes *Poppins* (para títulos) e *Roboto* (para textos corridos).

- **Bootstrap 5.3.3 CSS**: Carrega o framework CSS via CDN (Content Delivery Network), fornecendo utilitários de grid, formulários, botões e classes de layout responsivo.
- **Bootstrap Icons**: Importa a biblioteca vetorial de ícones (como a lupa de pesquisa e o calendário).

2. Estilos Customizados (<style>)

```CSS
  :root {
    --cor-destaque: #f75815;
    --cor-titulo: #2d465e;
    --cor-fundo: #f9fbfd;
  }
```

- **:root**: Define **variáveis de CSS nativas**. Centralizar cores como --cor-destaque permite alterar a paleta visual do site inteiro modificando apenas uma linha.

```CSS
  body {
    font-family: 'Roboto', sans-serif;
    background-color: var(--cor-fundo);
    color: #444444;
  }

  h1, h2, h3, h4 {
    font-family: 'Poppins', sans-serif;
    color: var(--cor-titulo);
  }
```

- Aplica a fonte padrão e o tom cinza-claro de fundo a toda a página, além de atribuir a fonte *Poppins* e a cor azul-escura a todos os títulos (h1 a h4).

```CSS
  .logo-site {
    font-size: 1.9rem;
    font-weight: 700;
    color: var(--cor-titulo);
    text-decoration: none;
  }
```

```
.logo-site span {
  color: var(--cor-destaque);
}
```

- Estiliza o logotipo textual. O span interno recebe a cor laranja de destaque para colorir o ponto final (BlogPy.).

```css
.botao-menu {
  color: var(--cor-titulo);
  text-decoration: none;
  padding: 6px 16px;
  border-radius: 20px;
  font-weight: 500;
  transition: 0.2s;
}

.botao-menu:hover,
.botao-menu.ativo {
  background-color: var(--cor-destaque);
  color: #ffffff;
}
```

- Cria botões arredondados em formato de pílula para o menu de categorias.
- O estado :hover e a classe .ativo preenchem o botão com fundo laranja e texto branco.

```css
.cartao-artigo {
  background: #ffffff;
  border: 1px solid #e6e9ec;
  border-radius: 12px;
  overflow: hidden;
  height: 100%;
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.cartao-artigo:hover {
  transform: translateY(-4px);
```

```
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.08);
  }
```

- **.cartao-artigo**: Define a caixa de cada post como um card com cantos arredondados (border-radius: 12px).
- **:hover**: Cria uma animação suave elevando o card em 4 pixels para cima (translateY(-4px)) e projetando uma sombra suave ao passar o cursor.

CSS
```
  .cartao-artigo img {
    width: 100%;
    height: 200px;
    object-fit: cover;
  }
```

- Garante que todas as fotos dos artigos tenham exatamente 200px de altura. O comando **object-fit: cover** recorta e centraliza a imagem proporcionalmente, impedindo distorções visuais.

CSS
```
  .etiqueta-categoria {
    background-color: rgba(247, 88, 21, 0.12);
    color: var(--cor-destaque);
    font-size: 0.78rem;
    font-weight: 600;
    padding: 4px 10px;
    border-radius: 20px;
    text-transform: uppercase;
  }

  .btn-laranja {
    background-color: var(--cor-destaque);
    color: #ffffff;
    border: none;
  }

  .btn-laranja:hover {
    background-color: #d8470a;
    color: #ffffff;
```

```
}
```

- **.etiqueta-categoria**: Tag visual suave (laranja translúcido) usada sobre os títulos dos posts.
- **.btn-laranja**: Botão primário do site, com escurecimento de cor no :hover.

3. Cabeçalho, Busca e Menu de Categorias (<header>)

```HTML
<header class="bg-white border-bottom shadow-sm">
  <div class="container py-3 d-flex flex-wrap
justify-content-between align-items-center gap-3">
    <a href="/" class="logo-site">BlogPy<span>.</span></a>
```

- **<header>**: Contêiner semântico do topo. Utiliza classes utilitárias do Bootstrap:
  - container: Centraliza o conteúdo com margens responsivas.
  - d-flex flex-wrap justify-content-between align-items-center: Ativa o Flexbox para alinhar a logo à esquerda e a barra de busca à direita, permitindo quebra de linha em telas pequenas (flex-wrap).
- O link do logotipo aponta diretamente para a rota raiz (/).

```HTML
    <form action="/" method="GET" class="d-flex">
      <div class="input-group">
        <input
          type="text"
          name="busca"
          value="{{ termo_busca }}"
          class="form-control"
          placeholder="Pesquisar artigo...">
        <button type="submit" class="btn btn-laranja">
          <i class="bi bi-search"></i>
        </button>
      </div>
    </form>
  </div>
```

- **Formulário de Busca**:
  - action="/" e method="GET": Ao enviar, os dados viram parâmetros na URL (ex: /?busca=python).

- name="busca": É o identificador lido pelo Python no app.py via request.args.get("busca").
- **value="{{ termo_busca }}"**: Expressão **Jinja2** que reinsere o texto digitado anteriormente no campo após a página recarregar.
- : Ícone de lupa do Bootstrap Icons dentro do botão de envio.

```HTML
  <nav class="bg-light border-top py-2">
    <div class="container d-flex flex-wrap justify-content-center gap-2">
      <a href="/?categoria=Todas" class="botao-menu {% if categoria_atual == 'Todas' %}ativo{% endif %}">
        Todas
      </a>
      <a href="/?categoria=Python" class="botao-menu {% if categoria_atual == 'Python' %}ativo{% endif %}">
        Python
      </a>
      ...
```

- **Barra de Categorias (<nav>)**:
  - Cada link envia uma requisição GET com o parâmetro categoria na URL (ex: /?categoria=Python).
  - **{% if categoria_atual == '...' %}ativo{% endif %}**: Lógica condicional do Jinja2. Se a variável categoria_atual enviada pelo Flask coincidir com o nome do botão, a classe CSS ativo é inserida na tag, destacando o botão com a cor laranja.

4. Mensagens do Sistema (Sucesso e Erro 404)

```HTML
<main class="container my-5">
  {% if aviso_sucesso %}
  <div class="alert alert-success d-flex align-items-center mb-4">
    <i class="bi bi-check-circle-fill me-2"></i>
    <span>{{ aviso_sucesso }}</span>
  </div>
  {% endif %}
```

- **<main>**: Identifica a área de conteúdo principal da página.

- **{% if aviso_sucesso %}**: Bloco condicional do Jinja2. Se aviso_sucesso contiver uma mensagem de texto (enviada após o usuário comentar), o HTML renderiza uma barra de alerta verde (alert alert-success). Se for None (como na rota inicial), todo esse bloco é ignorado.

```HTML
  {% if erro_404 %}
  <section class="p-5 mb-5 text-center bg-white border rounded-4
shadow-sm">
    <h1 class="display-4 fw-bold text-danger">404</h1>
    <h2 class="h4">Ops! Esta página não foi encontrada</h2>
    <p class="text-muted">O endereço digitado não existe no
servidor Python.</p>
    <a href="/" class="btn btn-laranja mt-2">Voltar para o
Início</a>
  </section>
  {% endif %}
```

- **{% if erro_404 %}**: Ativado apenas quando o decorador @aplicacao.errorhandler(404) do Flask dispara erro_404=True, exibindo uma mensagem amigável de erro no lugar de uma tela em branco do servidor.

5. Seção de Artigos com Laço for do Jinja2

```HTML
  <section class="mb-5">
    <div class="d-flex justify-content-between align-items-center
mb-4">
      <h2 class="h4 fw-bold mb-0">
        Artigos Publicados ({{ postagens|length }})
      </h2>

      {% if termo_busca or categoria_atual != 'Todas' %}
      <a href="/" class="btn btn-outline-secondary
btn-sm">Mostrar Tudo</a>
      {% endif %}
    </div>
```

- **{{ postagens|length }}**: Filtro do Jinja2 equivalente à função len(postagens) do Python. Exibe a contagem numérica de quantos artigos estão na lista no momento.

- **{% if termo_busca or categoria_atual != 'Todas' %}**: Exibe o botão "Mostrar Tudo" apenas se o usuário tiver feito uma busca ou filtrado por alguma categoria específica, permitindo resetar os filtros voltando para /.

```HTML
    <div class="row g-4">
      {% for post in postagens %}
      <div class="col-md-4">
        <article class="cartao-artigo d-flex flex-column">
          <img src="{{ post.imagem }}" alt="{{ post.titulo }}"
class="img-fluid">

          <div class="p-4 d-flex flex-column flex-grow-1">
            <div class="mb-2">
              <span class="etiqueta-categoria">{{ post.categoria
}}</span>
            </div>

            <h3 class="h5 fw-bold">{{ post.titulo }}</h3>
            <p class="text-muted small flex-grow-1">{{
post.resumo }}</p>

            <div class="border-top pt-3 mt-2 d-flex
justify-content-between small text-muted">
              <span><i class="bi bi-person me-1"></i>{{
post.autor }}</span>
              <span><i class="bi bi-calendar3 me-1"></i>{{
post.data }}</span>
            </div>
          </div>

        </article>
      </div>
```

- **Grid Responsivo (row g-4 + col-md-4)**: Divide o espaço em 12 colunas imaginárias. Como cada coluna ocupa 4 unidades no desktop (md), exatamente 3 artigos cabem em cada linha ($12 \div 4 = 3$). Em celulares, os cards ocupam a largura inteira (12 unidades).
- **{% for post in postagens %}**: Itera sobre a lista de dicionários enviada pelo Flask, replicando o bloco para cada item.
- **{{ post.imagem }}, {{ post.titulo }}, etc.**: Sintaxe de interpolação do Jinja2 que lê diretamente as chaves de cada dicionário do app.py.

- **flex-grow-1**: Garante que o texto do resumo e o corpo do card se expandam igualmente, mantendo o rodapé do card (autor e data) sempre alinhado na parte inferior.

```HTML
      {% else %}
      <div class="col-12">
        <div class="p-4 text-center bg-white border rounded-3">
          <p class="mb-2">Nenhum artigo encontrado para sua
pesquisa.</p>
          <a href="/" class="btn btn-laranja btn-sm">Ver todos os
artigos</a>
        </div>
      </div>
      {% endfor %}
    </div>
  </section>
```

- **{% else %} dentro do {% for %}**: Recurso especial do Jinja2 executado quando a lista postagens está vazia (por exemplo, quando a busca não encontrou nenhum resultado).
- **{% endfor %}**: Finaliza o bloco de repetição.

6. Mural de Comentários e Formulário POST

```HTML
  <section class="bg-white p-4 rounded-4 border shadow-sm">
    <div class="row g-4">
      <div class="col-md-6">
        <h3 class="h5 fw-bold mb-3">
          <i class="bi bi-chat-dots me-1"></i> Mural da Turma ({{
comentarios|length }})
        </h3>

        {% for item in comentarios %}
        <div class="p-3 mb-2 bg-light border rounded-3">
          <strong class="d-block text-dark">{{ item.nome
}}</strong>
          <span class="text-secondary small">{{ item.mensagem
}}</span>
        </div>
        {% endfor %}
```

```
      </div>
```

- Divide a seção em duas metades com col-md-6 ($12 \div 2 = 6$).
- **Coluna da Esquerda**: Itera sobre a lista COMENTARIOS com {% for item in comentarios %}, exibindo o nome de quem comentou em negrito e a mensagem logo abaixo.

HTML
```
      <div class="col-md-6">
        <h3 class="h5 fw-bold mb-3">Deixe seu Comentário</h3>
        <form action="/comentar" method="POST">
          <div class="mb-3">
            <label class="form-label small fw-bold">Seu
Nome:</label>
            <input type="text" name="nome" class="form-control"
placeholder="Ex: João Pedro" required>
          </div>

          <div class="mb-3">
            <label class="form-label small fw-bold">Sua
Mensagem:</label>
            <textarea name="mensagem" rows="2"
class="form-control" placeholder="Escreva seu comentário..."
required></textarea>
          </div>

          <button type="submit" class="btn btn-laranja w-100">
            <i class="bi bi-send me-1"></i> Publicar Comentário
          </button>
        </form>
      </div>
    </div>
  </section>
</main>
```

- **Coluna da Direita (Formulário)**:
  - action="/comentar": Endereço para onde os dados são despachados.
  - method="POST": Envia as informações no corpo da requisição HTTP (e não expostas na URL).

- name="nome" e name="mensagem": Nomes dos campos capturados no Flask por request.form.get("nome") e request.form.get("mensagem").
- required: Atributo nativo do HTML5 que impede o envio do formulário se o campo estiver em branco antes mesmo da requisição chegar ao servidor.
- w-100: Faz o botão ocupar 100% da largura da coluna.

7. Rodapé (<footer>)

```HTML
  <footer class="bg-white border-top py-4 mt-5 text-center small text-muted">
    <div class="container">
      <p class="mb-1">
        <strong>BlogPy</strong> — Template Didático (HTML5 + CSS3 + Bootstrap 5 + Python/Flask).
      </p>
      <p class="mb-0">
        Quer testar a função de erro do Python? <a href="/pagina-que-nao-existe">Clique aqui para simular o Erro 404</a>.
      </p>
    </div>
  </footer>

</body>
</html>
```

- **<footer>**: Seção semântica de rodapé, com fundo branco e texto centralizado em tom atenuado (text-muted).
- **Link de teste**: Aponta propositalmente para /pagina-que-nao-existe, permitindo testar a função @aplicacao.errorhandler(404) configurada no arquivo Python.
- As tags </body> e </html> fecham o corpo e o documento HTML.